

Starting Now >

Design Production-Ready Power BI & Fabric Analytics Systems (Workshop)

A hands-on workshop for architects, BI engineers, and technical leaders accountable for scalable Power BI & Microsoft Fabric solutions.

Greg Deckler



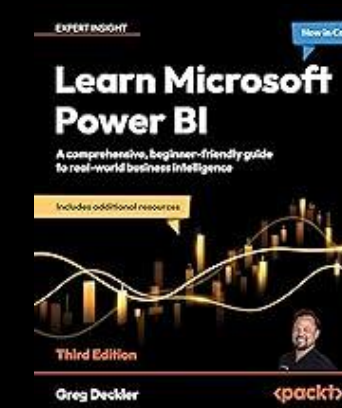
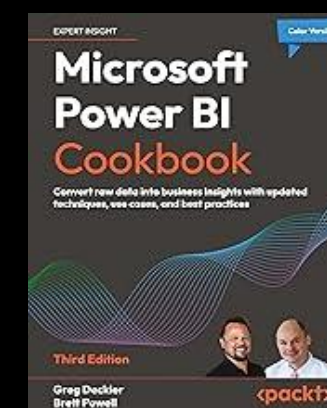
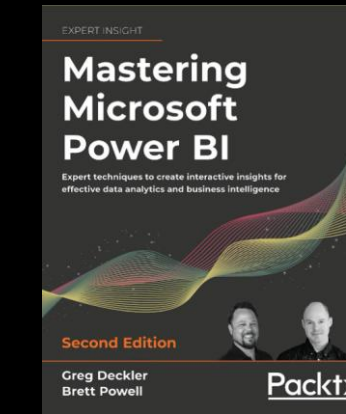
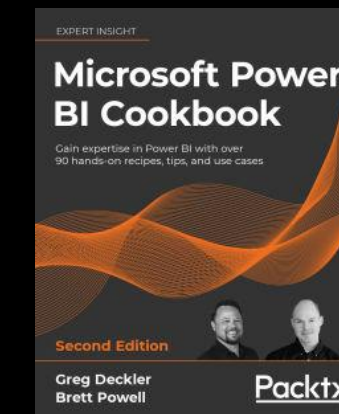
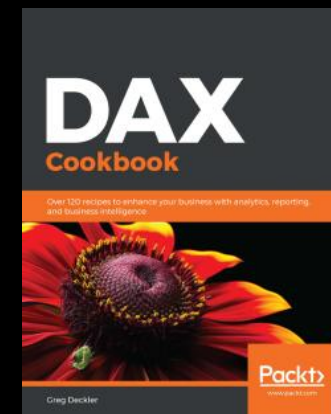
Design Production-Ready Power BI & Fabric Analytics Systems



Greg Deckler

About Greg Deckler

- 32 years of IT consulting expertise
- 7-time Microsoft MVP
- 9 books published
- Email: gdeckler@gmail.com
- Workshop Materials:
https://github.com/PacktPublishing/Learn-Microsoft-Power-BI_3E/tree/main/Workshop
- LinkedIn:
<https://www.linkedin.com/in/gregdeckler/>
- X: @gregdeckler
- Bluesky: @gregdeckler.bsky.social
- YouTube:
 - <https://www.youtube.com/@microsoftthatesgreg>
 - <https://www.youtube.com/@daxforhumans>



Agenda

-
- 01 Welcome & Orientation

 - 02 Systems thinking: Reports vs. analytics systems

 - 03 Fabric architecture patterns that hold up in enterprise

 - 04 Break

 - 05 Semantic modeling at scale

 - 06 Capacity, cost, and governance without killing agility

 - 07 Production readiness framework

 - 08 Q&A + next steps



Section 1: Welcome & Orientation

- Introductions
- What are you hoping to get out of this workshop?
- Participation is key!

Section 2: Systems thinking: Reports vs. analytics systems

Systems Thinking: Reports vs Analytics Systems

From dashboard-building to
semantic model and architecture
design in Power BI / Microsoft
Fabric

Case study

Hands-on diagnosis

Architecture redesign

Scenario

A retailer scales from a few dashboards to a fragile analytics estate.

Tension

Performance, flexibility, cost, and maintainability are all in conflict.

Lesson

The product is the semantic model and system design - not the report page.

The case: a retailer that outgrew “just build a dashboard”

What started as one sales report became a fragmented Fabric / Power BI ecosystem.

Business context

- 250 stores
- E-commerce sales
- ERP for inventory and purchasing
- CRM for promotions and loyalty
- Daily executive reporting
- Weekly merchant analysis
- Store-level operational reporting

What the team now has

- 40+ reports
- Multiple duplicated semantic models
- Mixed Import, DirectQuery, and Direct Lake
- Different KPI definitions by team
- Rising capacity costs and slow pages

How the problem evolved

1

Sales dashboard

Fast initial win for leadership.

2

More team-specific reports

Finance, merchandising, stores each ask for variations.

3

Local logic everywhere

Measures, date logic, and joins copied into many PBIX files.

4

Architecture debt shows up

Costs rise, KPIs conflict, refreshes drift, performance falls.

Systems thinking reframing

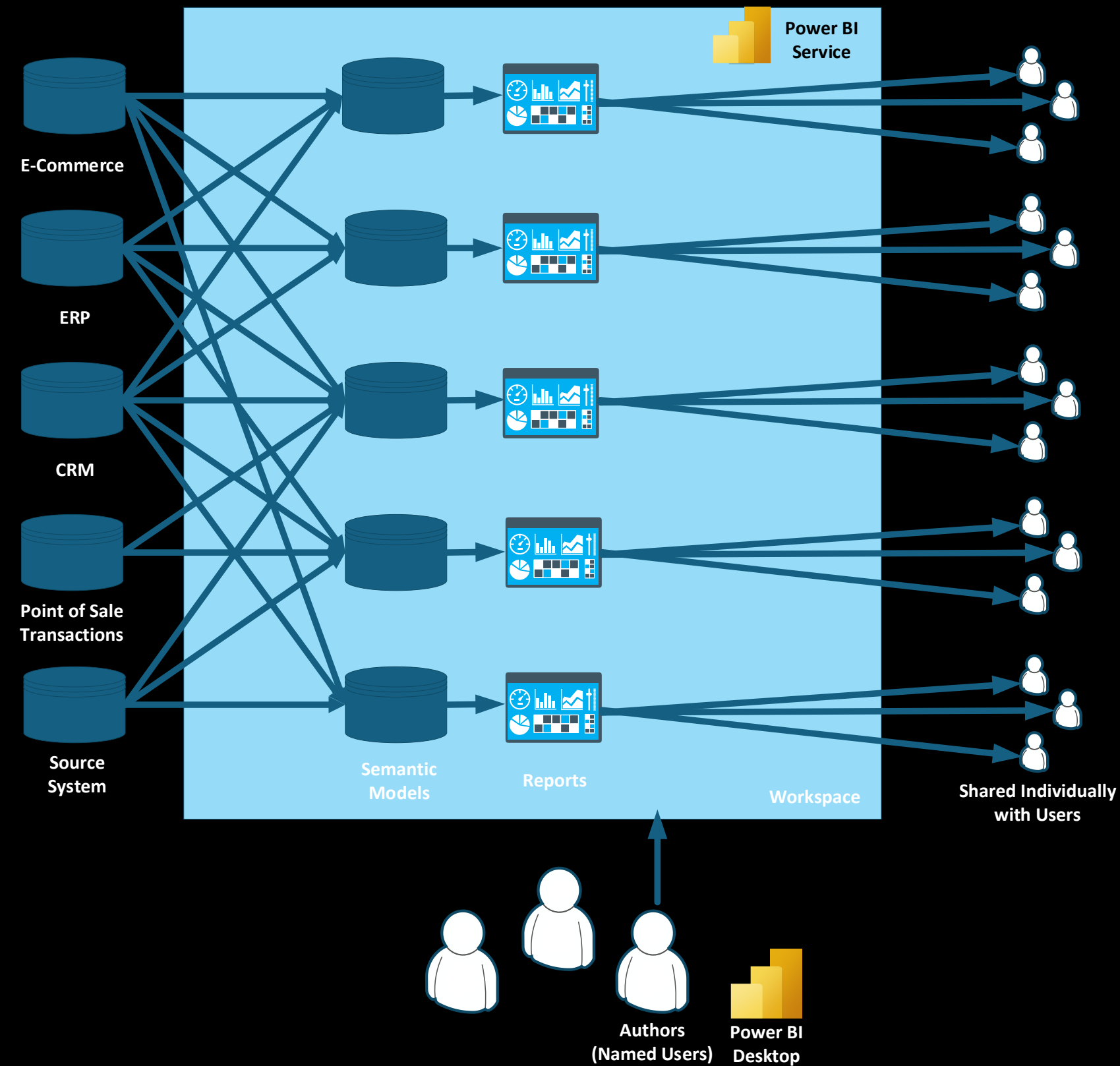
The report/dashboard is not the product.

The semantic model is the reusable product surface, and the architecture is what determines whether the whole analytics system stays fast, trusted, and affordable.

Shift: reports -> system

What the team now has

- 40+ reports
- Multiple duplicated semantic models
- Mixed Import, DirectQuery, and Direct Lake
- Different KPI definitions by team
- Rising capacity costs and slow pages
- 1 workspace
- 1 author publishing under their user account
- 1 report per semantic model
- Sharing to individual users
- Some users have View access to workspace to be able to view all reports
- Long lead times for new reports



Hands-on diagnostic exercise

Use the broken architecture to drive a systems-level discussion, not a page-design critique.

Prompt

The retailer says:

- Reports are slow
- KPIs do not match
- Costs are rising
- Every new dashboard takes too long

Diagnose the architecture and recommend a better system design.

Goal: Find the system failures

Ask participants to determine the following

- What belongs in the governed semantic model?
- Which workloads should be split apart?
- Where do Import, DirectQuery, or Direct Lake actually make sense?
- Which metric definitions need governance and ownership?
- Which facts and dimensions must be conformed?
- Which problems are technical vs organizational?



Systemic failures participants should identify

Each symptom maps back to a design failure, not just a bad report page.

Decentralized KPI logic	“Net Sales” and margin definitions vary by report and team.	No conformed dimensions	Product, store, customer, date, and channel are inconsistent across models.
Semantic model treated as a byproduct	Visuals were prioritized; the reusable metric layer was not.	Refresh strategy misaligned to business value	The business pays for freshness where it is not needed.
Storage modes chosen without workload design	DirectQuery is used because data is “big,” not because latency demands it.	No workload separation	Exec, operational, and exploratory use cases all compete on the same capacity.
Mixed grains without explicit modeling	Transactions, inventory snapshots, promos, and monthly budgets are combined awkwardly.	Report sprawl hides ownership failure	Nobody owns the semantic model as a product with standards and change control.



Section 3: Fabric architecture patterns that hold up in enterprise

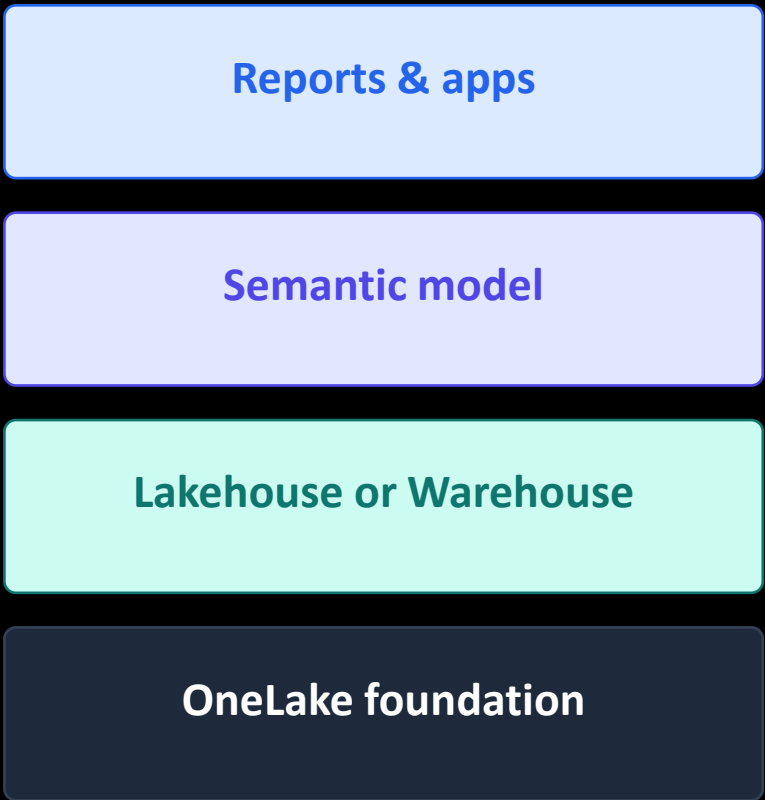


Fabric architecture
patterns that hold up
in enterprise

Lakehouse vs Warehouse vs Semantic Model roles
Ingestion choices • Direct Lake fit • Hands-on architecture design

ENTERPRISE PLAYBOOK

Microsoft Fabric / OneLake

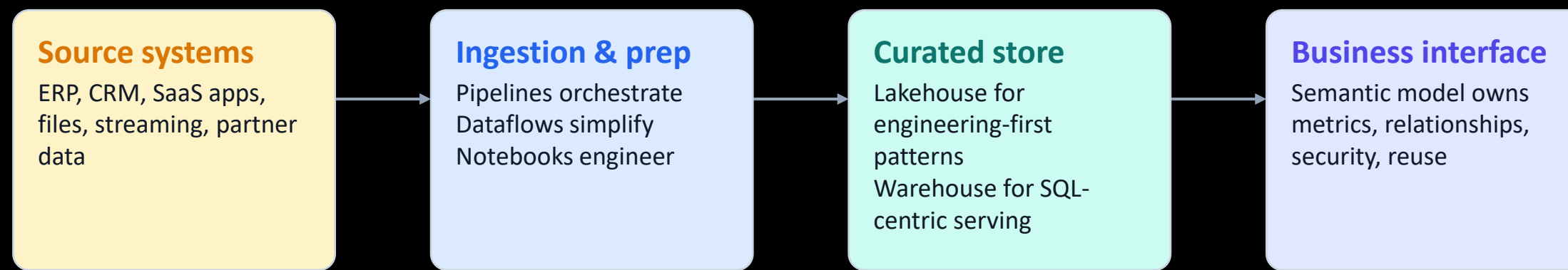


Clear role separation beats
workspace-by-workspace sprawl



The enterprise pattern: separate responsibilities, not just tools

Treat OneLake as the shared substrate, treat data stores and semantic models as distinct roles.



Why this holds up

- Fewer copies of data
- Reusable pipelines and shared dimensions
- Less metric drift across departments
- Clear ownership boundaries from engineering to BI

What to avoid

- Every workspace builds its own lake + model
- Semantic logic buried inside reports
- Direct Lake used as a shortcut around architecture
- Low-code sprawl without orchestration or standards

Role clarity: Lakehouse vs Warehouse vs Semantic Model

Use each item for the job it is best at, do not let every layer become a reporting endpoint.

Lakehouse

Best for:

- Delta tables in OneLake
- Spark / notebook engineering
- Medallion data design
- Mixed structured + semi-structured data
- Reuse across analytics and data science

Use when the operating model is engineering-first.

engineering-first

Warehouse

Best for:

- T-SQL-centric teams
- Curated relational serving
- Dimensional models and conformed marts
- BI-facing schemas with tighter SQL control
- Classic EDW migration paths

Use when the operating model is SQL-first.

sql-first

Semantic Model

Best for:

- Measures and KPI definitions
- Relationships and hierarchies
- Row-level security
- Certified reusable business logic
- Consistent self-service consumption

Use as the governed business contract.

business-first



Ingestion choices: who does what?

A simple rule set helps teams avoid using one Fabric item for every problem.

Source data

SaaS • DBs
files • streams

Dataflows Gen2

Low-code shaping

Good for quick source prep, analyst-friendly transforms, and moderate complexity.

Do not make it your entire enterprise engineering strategy.

Pipelines

Orchestration

Good for schedules, dependencies, retries, branching, and coordinating copy activities, notebooks, and downstream refresh.

Notebooks

Code-first engineering

Good for Spark, scale, advanced transformation logic, reusable engineering, ML adjacency, and heavy data quality work.

Decision rule

Use Dataflows for low-code shaping, Pipelines for orchestration, and Notebooks when complexity, scale, or engineering rigor justifies code. Most enterprise solutions use all three with Pipelines coordinating the flow.



Where Direct Lake fits and where it does not

Direct Lake is a fast path for curated Fabric-native data, not a replacement for architecture discipline.

Use Direct Lake when...

- Data is already curated in OneLake-backed Delta tables
- Large or frequently updated data makes full import awkward
- You still want semantic-model speed and governed business logic
- You are building an IT-driven or shared analytics layer
- Capacity, guardrails, and design validated in a POC

Do not reach for it when...

- Source data is still messy or unstable
- You need ad hoc mashups across many non-Fabric sources
- Teams expect it to eliminate the semantic layer
- SQL views / unsupported features cause fallback behavior you have not planned for
- Capacity and governance constraints are still unknown

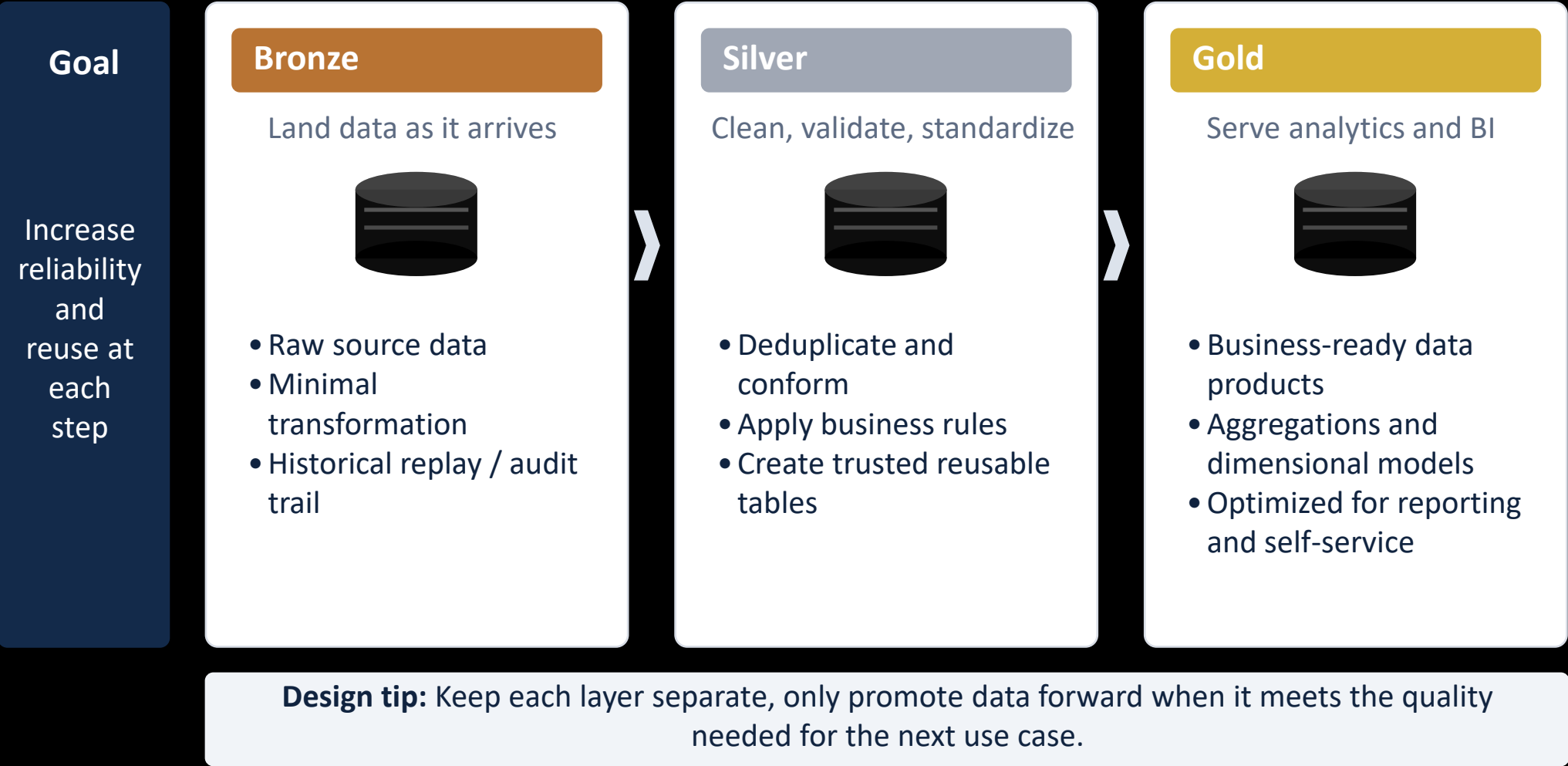
Best home: the gold / curated analytics layer



Medallion architecture pattern

A layered design that progressively improves data quality from raw ingestion to business-ready analytics

Think: Raw -> Refined -> Ready



Patterns that usually hold up in enterprise

Pick one as the dominant operating model, do not mix patterns accidentally.

1. Central platform + department semantic products

Shared engineering workspace(s), common conformed dimensions, certified enterprise measures, and department-specific semantic models for local reporting. Best when governance and cross-functional KPIs matter most.

2. Lakehouse-first medallion + curated business layer

Bronze/silver/gold in Lakehouse, notebooks for heavier engineering, semantic models over gold data, and Direct Lake only on curated layers that justify it. Best when engineering and data science are close partners.

3. Warehouse-first curated BI pattern

Lakehouse or staging for ingestion, Warehouse for dimensional serving, and a shared semantic model over conformed marts. Best when teams are SQL-centric or migrating classic EDW practices into Fabric.

Anti-patterns and guardrails

These are the design habits that usually create rework, duplicate data, and inconsistent metrics.

Anti-patterns

- One workspace per department with no shared model
- Every team picks its own store with no standards
- Semantic logic lives inside reports
- Direct Lake gets chosen because it sounds modern
- Dataflows become the only integration pattern

Patterns

- Shared definitions for facts, dimensions, and KPIs
- Reusable ingestion templates and orchestration
- Security and environment separation by design
- Certified semantic models for enterprise metrics
- Direct Lake only after a capacity-aware proof of concept

A useful governance rule

Give departments autonomy at the report and local semantic-product layer and standardize the curated data contracts underneath. That preserves self-service without turning the platform into a set of isolated mini-stacks.

Hands-on scenario: design a multi-department Fabric architecture

Challenge teams to make explicit choices under realistic constraints.

Departments

Sales

- Near-real-time dashboards
- Self-service analysis

Marketing

- Certified metrics
- Controlled change

Operations

- High-volume telemetry / logs
- Engineering-heavy transforms

Constraints

- Executives want cross-department KPIs
- Budget limits duplicate storage
- Central IT wants reusable pipelines
- Dev / test / prod promotion is required
- Department teams still want autonomy

Team outputs

Ask each group to decide:

1. Where raw data lands
2. Where curated data lives
3. Which items are shared vs isolated
4. Where semantic logic belongs
5. Whether Direct Lake is justified and why

Evaluation criteria

A strong design is one that can explain its choices, trade-offs, and boundaries.

Questions every team should answer

1. Where does raw data land?
2. Where is business-ready data created?
3. Which layer owns conformed dimensions?
4. Where do metric definitions live?
5. What is shared across departments?
6. What is isolated for security or ownership?
7. Why use Direct Lake here and not elsewhere?
8. How many copies of the same data exist, and why?

Good final answer sounds like...

“OneLake is shared, curation is standardized, the semantic layer is governed, and each department gets autonomy only where it adds value. Direct Lake is used selectively on curated Fabric-native data, not as a shortcut around modeling, governance, or capacity planning.”

Fabric Architecture Design Playbook

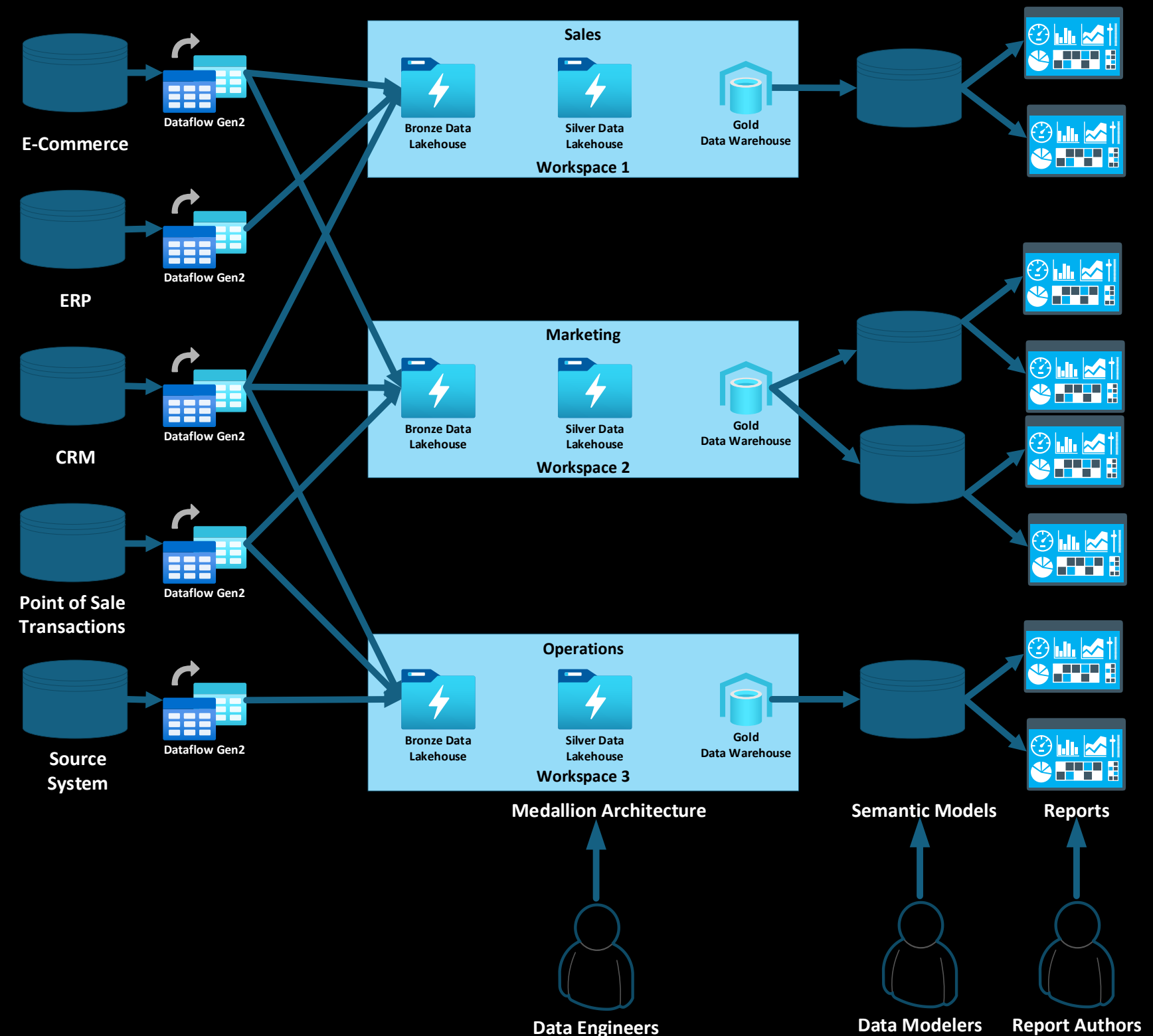
Target-state redesign

A better answer is an analytics system with a governed semantic layer and workload-aware architecture.

Faster
Trusted
Cheaper
Easier to evolve

Key takeaway

The design question is not “what dashboard should we build next?” It is “what semantic model and system architecture let many good dashboards emerge with consistent metrics, acceptable cost, and sustainable maintenance?”



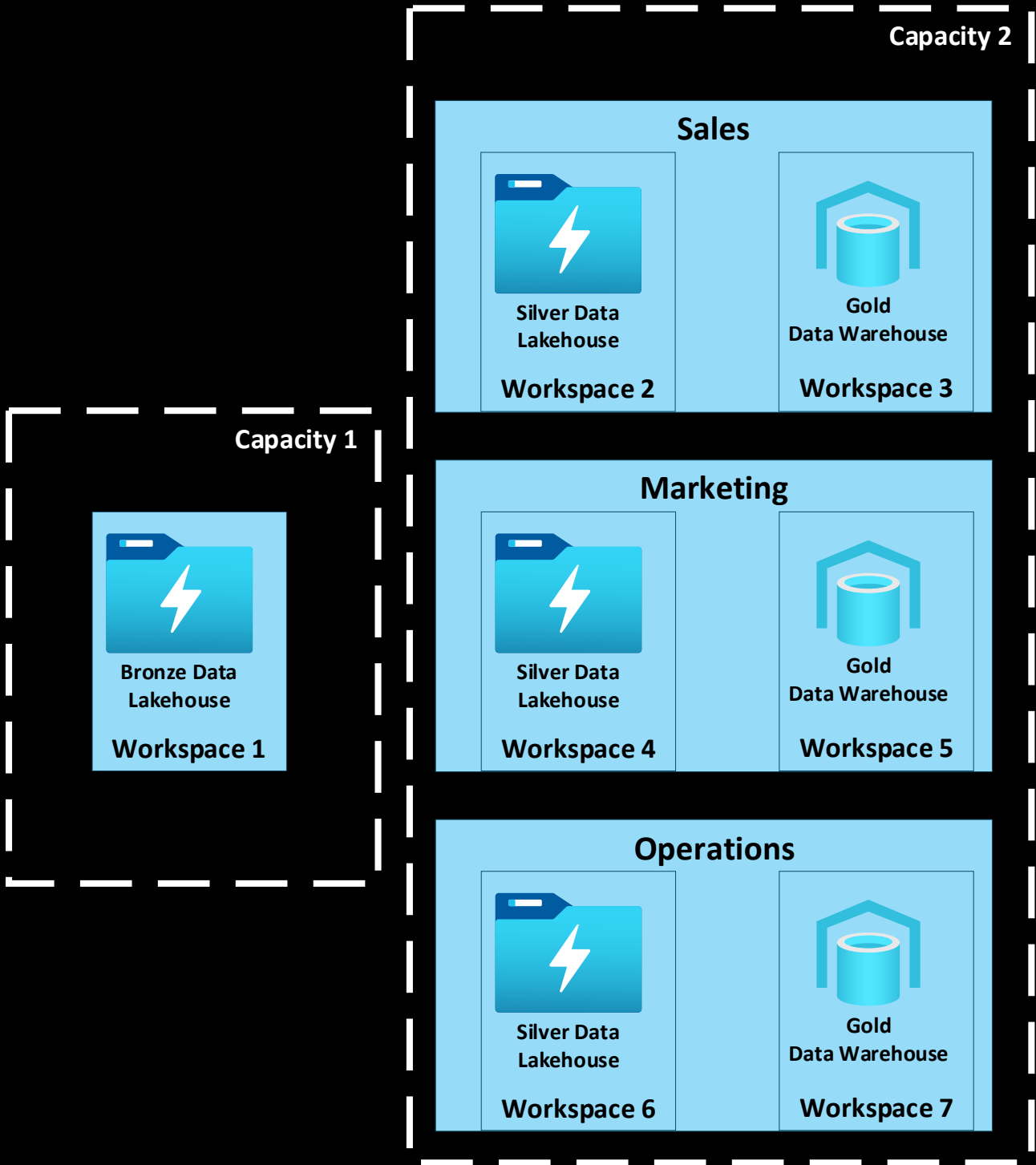


Alternatives

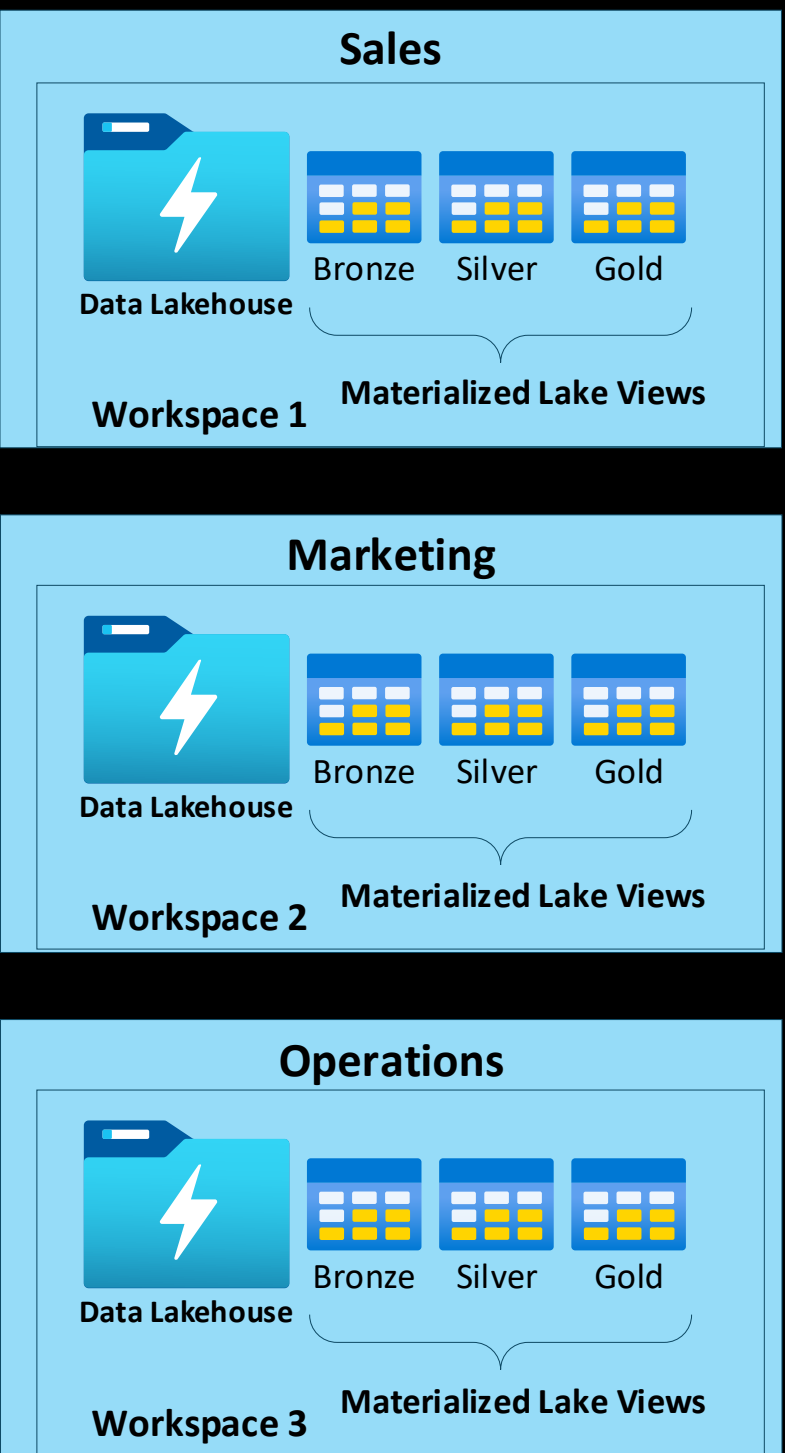
Alternative 1



Alternative 2



Alternative 3



Section 4: Break

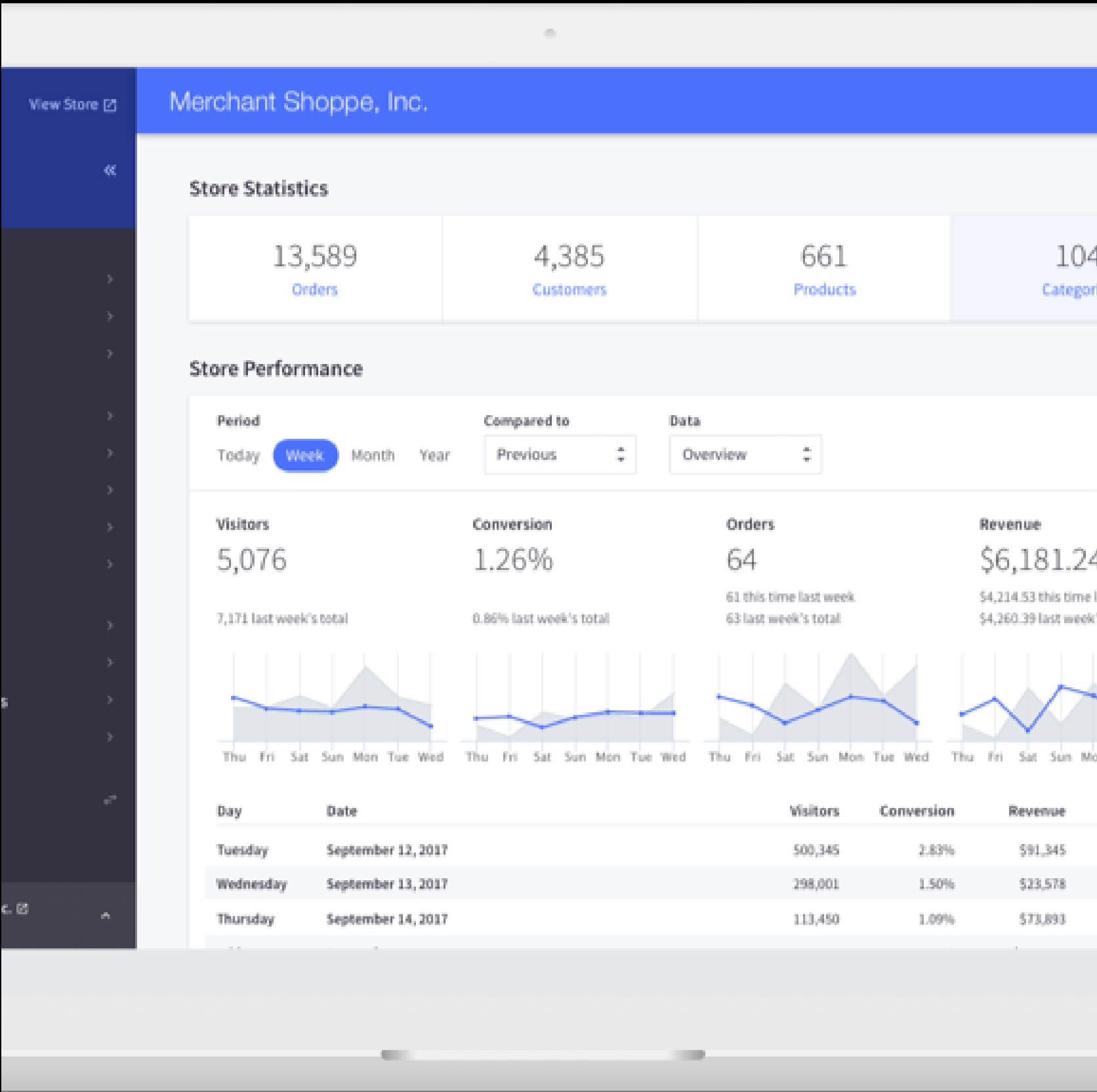
Section 5: Semantic modeling at scale



Semantic modeling at scale

What actually drives performance — and how to redesign a model for growth, refresh, and concurrency

Star vs flat vs hybrid • VertiPaq compression • Incremental refresh + hybrid tables • DAX patterns that scale



What actually drives performance

At scale, performance is rarely “just a DAX problem.”

Model shape

Fact grain, dimensions, relationships, filter paths

Storage mode

Import vs DirectQuery vs Dual vs hybrid tables

VertiPaq cost

Cardinality, text-heavy columns, timestamps, GUIDs

Refresh design

Partitions, folding, incremental windows, freshness

Query behavior

Concurrency, visual patterns, measure evaluation paths

Operational fit

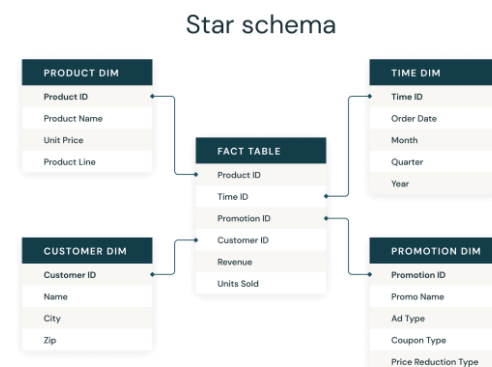
Memory pressure, source load, capacity choices

Practical rule: the fastest model is the one that keeps common queries in-memory, keeps filters simple, and refreshes only the data that changed.

Star vs flat in the real world

Star schema

- Best default for scale, reuse, and predictable filtering
- Cleanest DAX because dimensions filter and facts summarize
- Easiest to govern as row counts and users grow



Flat model

- Can feel quick early when the model is small
- Repeated attributes increase size and maintenance pain
- Usually gets harder to evolve as logic and concurrency rise

Bottom line:

- Star schema is the safest foundation for good performance.



VertiPaq compression: why cardinality matters

Columns that usually cost the most

GUID / transaction ID	Very high cardinality, weak compression
Free-text description	Large dictionaries, low reuse
Precise timestamps	Near-unique values per row
Repeated text in facts	Same business meaning stored many times
Unneeded calc columns	Extra storage with no query payoff

Modeling moves that usually help

- Push row-level shaping upstream whenever possible
- Split date from time if time-of-day analysis is not needed everywhere
- Move descriptive text into dimensions instead of repeating it in facts
- Drop unused columns early, especially text, IDs, and debugging fields
- Keep fact tables numeric and narrow when you can

Simple message: not all columns cost the same. Compression quality is a design choice.



VertiPaq deep dive

How tables are stored

- **Partitions → Segments**
 - Each table is split into one or more *partitions* (usually one by default).
 - Each partition is split into **segments** of ~1M rows. Segment stats (min/max, distinct count, sort info) let VertiPaq skip whole chunks (“segment elimination”) during queries.
- **Columnar storage**
 - Columns are stored separately. A scan of a few columns never touches the others.
- **Encodings & compression**
 - **Dictionary encoding (most common):**
 - Per column, VertiPaq builds a **dictionary** of unique values.
 - The column’s data becomes a stream of **value IDs** (integers) that reference that dictionary.
 - IDs are **bit-packed** (fewest bits possible).
 - Runs of repeated IDs can be **run-length encoded (RLE)**. A A A B B C => A,3 B2, C1
 - **Value encoding (for numeric columns with many unique values):**
 - Stores scaled integers/floats directly, then bit-packs them (may skip a separate dictionary).
 - **Additional tricks:** sorting by a column can increase RLE effectiveness; low-cardinality columns compress extremely well.
- **No global dictionaries**
 - Each column has its **own** dictionary. The integer ID for “USA” in Table A’s Country column has nothing to do with the ID for “USA” in Table B’s Country column.

VertiPaq deep dive

How relationships are stored

- **Metadata + indexes, not pre-joined data**
 - A relationship record (one-to-many, direction, active/inactive, single/both direction, etc.) is stored as model metadata.
 - VertiPaq also creates **internal indexes** on the columns that participate in relationships (and on some other frequently filtered/grouped columns). Think of these as value→row locators optimized for fast lookups during filter propagation.
- **The hidden “blank row”**
 - In many-to-one relationships, the lookup table gets a hidden **blank row** to absorb fact-side keys that don’t match (the “unknown member”). This preserves DAX semantics when referential integrity isn’t perfect.



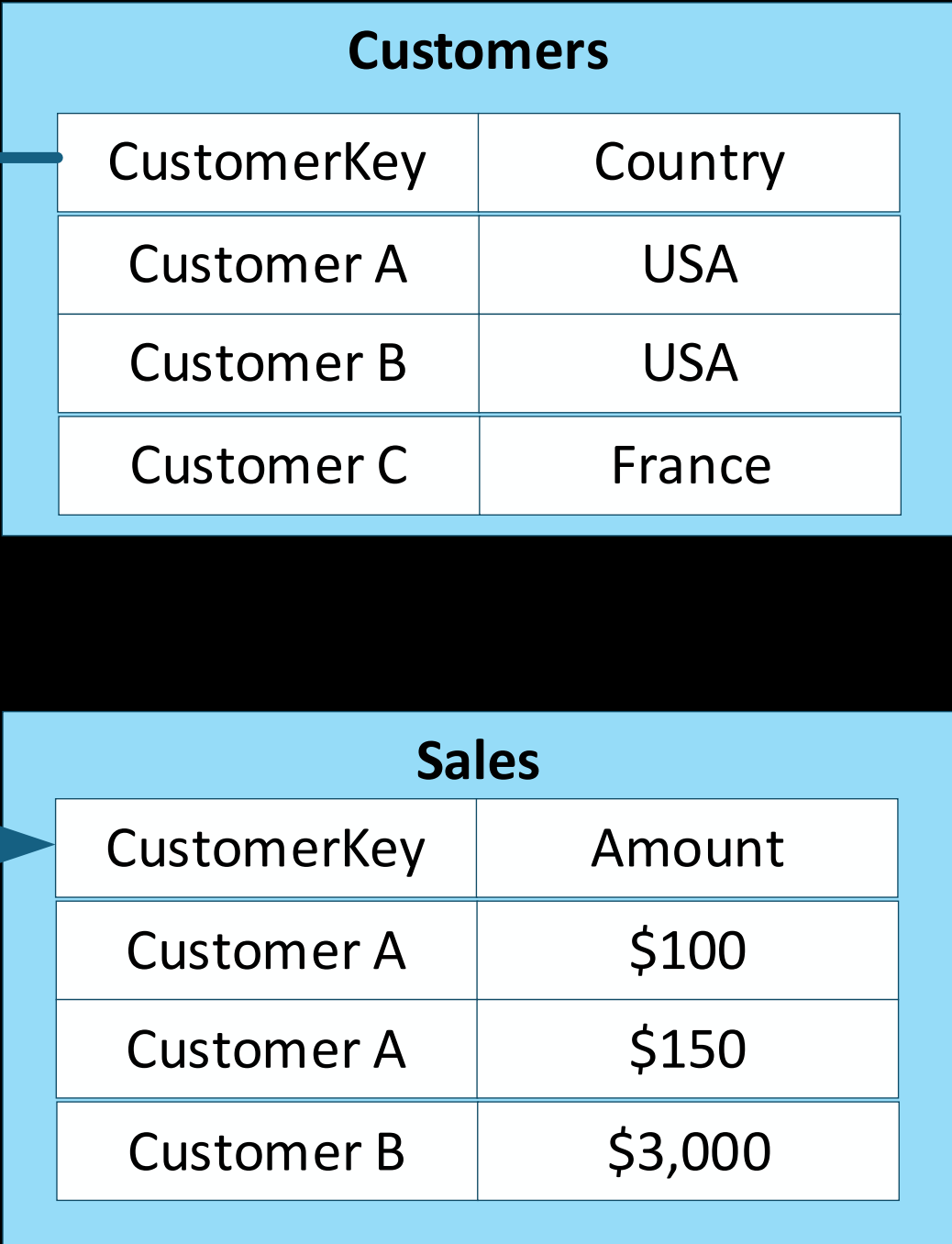
VertiPaq deep dive

What Actually Happens at Query Time (Filter Propagation)

Suppose you filter a **lookup** table (Customers[Country] = "USA") and there's a relationship to a **fact** table (Customers[CustomerKey] → Sales[CustomerKey]):

- **Filter on lookup**
VertiPaq resolves "USA" via **Customers[Country]**'s dictionary and applies a bitmap over matching **Customers** rows.
- **Translate to keys**
From the filtered Customers rows, it gets the relevant **CustomerKey** values using Customers[CustomerKey]'s dictionary/encodings.
- **Find fact rows**
Using the fact column's internal index on **Sales[CustomerKey]**, it maps those key values to the exact row IDs in **Sales**—often with segment elimination along the way.
- **Apply bitmap on the fact**
The result is a bitmap marking the Sales rows that survive the relationship filter.
No join table is materialized; it's all dictionary lookups, indexes, and bitmaps on compressed columns.

Because each column has its own dictionary, the engine **does not reuse IDs across tables**. Instead, it turns filtered values on the lookup side into **actual values**, then finds the corresponding **value IDs** within the fact column's *own* dictionary and applies those via its column index. That's the essence of relationship traversal in VertiPaq.



VertiPaq deep dive

Relationships cost something even though VertiPaq is in-memory

- A relationship traversal (e.g., Customers[CustomerKey] → Sales[CustomerKey]) adds **extra dictionary lookups and bitmap merges** at query time:
- The filter is applied to the lookup table's column dictionary.
- Those keys are translated to actual values.
- The fact table's index on its key column is used to find matching rows.
- A bitmap of valid rows is built/applied.

On 100 M rows, that's not "slow," but it's still work. A flat table simply skips all of that, your filters and aggregations hit one dictionary space directly.

VertiPaq deep dive

Flattening can improve *column locality* and *segment pruning*

- VertiPaq stores columns in **segments of ~1 M rows**.
When you flatten a model, attributes like Customer[Region], Product[Category], etc. become additional columns on the same table.
- If you filter Region = "Europe", VertiPaq can **eliminate entire Sales segments** immediately—because the segment stats are now on the same table.
- In a star schema, those same filters exist on separate lookup tables, and the engine must traverse relationships before it can know which fact rows to touch. That can reduce the effectiveness of segment elimination.

VertiPaq deep dive

Dictionary density and compression may improve

- A flat table often compresses *better* than expected because:
 - Repeated combinations of dimension attributes form *correlated patterns* VertiPaq can exploit.
 - Sorting (Power BI automatically sorts by surrogate keys or load order) can produce long runs of identical encoded IDs → **run-length encoding** gains.
 - Star schemas, on the other hand, have separate dictionaries for every dimension. Fact columns with surrogate keys are often high-cardinality integers that compress worse than repeated low-cardinality categorical values in a flattened structure.

VertiPaq deep dive

Less context transition and fewer filter propagations

- Many DAX measures implicitly trigger **context transitions** or **CALCULATE** operations that must propagate filters through relationships.
- When everything lives in one table, there's no propagation - just direct column filters - which can drastically simplify the internal query plan (fewer joins, smaller bitmaps, fewer hash lookups).

VertiPaq deep dive

Why flat tables can be faster

- Flat tables avoid relationship traversals.
- Segment pruning can be more efficient.
- Compression can overperform if the data is sorted or highly repetitive.
- Dictionary lookups are simpler (single dictionary space).
- DAX formulas avoid additional context propagation overhead.

In short: **VertiPaq's query pipeline is incredibly fast at scanning compressed columns, but filter propagation across relationships introduces real work.**

This is why a fully flattened, pre-joined table can outpace a perfectly modeled star schema.



VertiPaq deep dive

Flat vs. Star

Aspect	Flat table	Star schema
Compression ratio	Sometimes higher but limited by repeated dimension values	Usually better overall because small lookup tables compress independently
Model size	Grows with every new attribute	Minimal duplication
Scalability	Slower refresh & larger memory footprint when data grows beyond cache	Scales linearly, dimension reuse



Solving for scale and “freshness”

Aggregation tables vs. Incremental refresh vs. Hybrid tables

Aggregation tables

- Lower granularity for common queries
- Set within the semantic model for high granularity fact tables
- Can solve performance issues when viewing reports

Incremental refresh

- Common enterprise pattern
- Creates multiple partitions
- Can solve scaling problems in import models during data refresh

Hybrid tables

- Common enterprise pattern
- Import or Dual dimensions + very large or real-time facts
- Use when freshness and scale both matter

Bottom line:

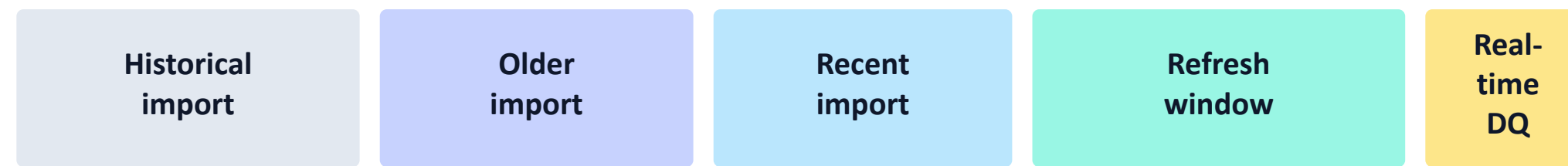
- Aggregation tables can solve many typical query performance issues caused by high granularity fact tables.
- Incremental refresh can help solve data refresh timing issues
- Hybrid tables can help resolve data latency or “freshness” issues

Incremental refresh + hybrid tables

Scale problems are often refresh and concurrency problems.

- 1 Incremental refresh partitions the table and refreshes only the hot window
- 2 Hybrid tables add a DirectQuery partition for the newest data

- 3 Use Dual dimensions so filters work efficiently against both import and DQ partitions
- 4 Query folding is non-negotiable for an incremental pattern to pay off



Outcome: shorter refresh windows, lower memory pressure, and better shared-user responsiveness.

DAX patterns that scale

Patterns that often hurt

- Heavy iterators over large fact tables
- Calculated columns for logic that belongs upstream
- Many-to-many + bidirectional filters as the default answer
- Broad ALL() patterns that wipe out useful filter context
- Measures written to compensate for a weak model

Patterns that usually hold up

- Measures on top of a clean star schema
- Explicit filter behavior and simple relationship paths
- Dimension-led slicing with fact-led aggregation
- Upstream transformations in Power Query or the warehouse
- Refactoring logic to reduce row-by-row work

Best takeaway: the fastest DAX is often the DAX you didn't need to write because the model was shaped correctly.

Hands-on modeling sprint: redesign a problematic model

Starting scenario

- One giant sales table with 300M+ rows
- Product, customer, and geography text repeated in the fact
- Bi-directional and many-to-many relationships everywhere
- Full refresh takes too long; slicers lag under load
- Business wants current-day visibility without losing speed

Sprint output

revised model diagram • storage mode plan • refresh design • 2–3 DAX refactors





Sprint workflow

1	1. Establish grain	What business event does one row represent?
2	2. Split the fact	Move descriptive attributes into dimensions
3	3. Remove expensive columns	Trim high-cardinality and unused fields
4	4. Simplify relationships	Prefer one-to-many, single-direction paths
5	5. Choose storage modes	Import, Incremental, Dual, DirectQuery, or Hybrid by table
6	6. Refactor measures	Rewrite the 2–3 worst scaling DAX patterns

Design decisions should be justified against three outcomes: model size, refresh time, and concurrent report responsiveness.

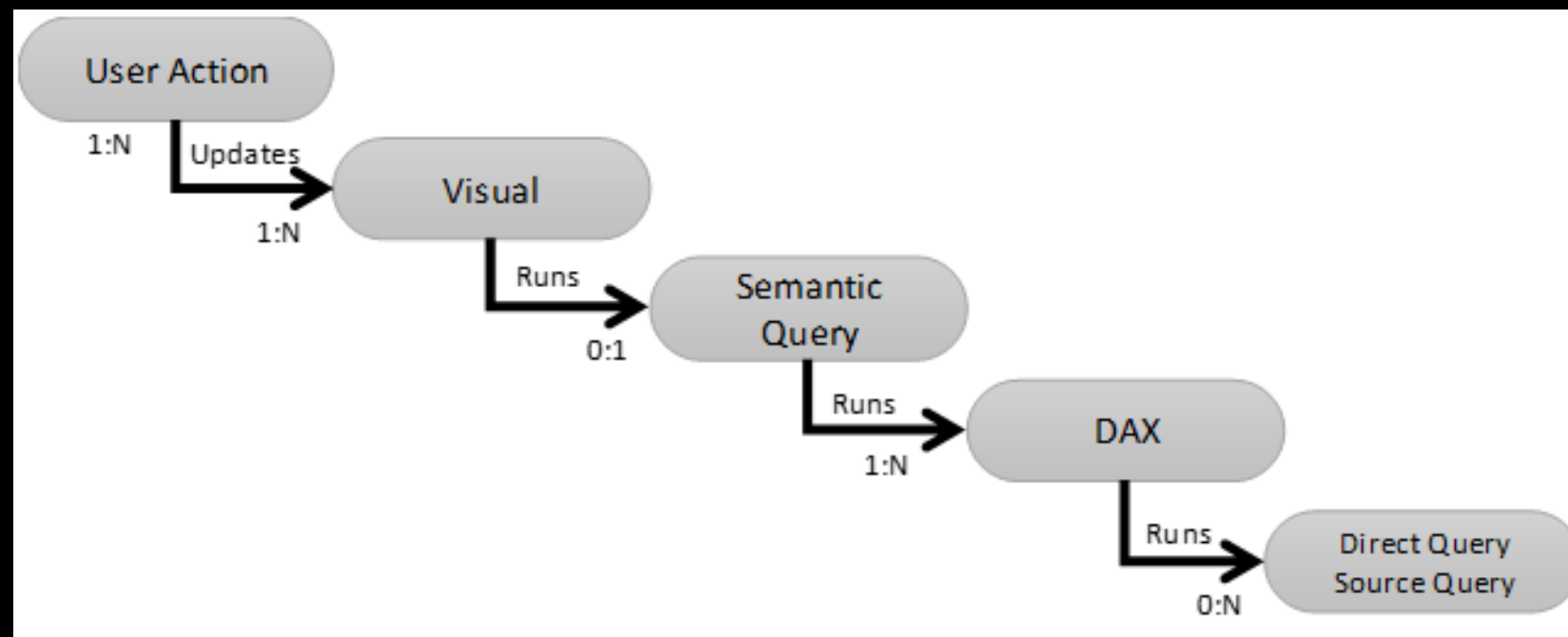


Don't ignore the front end

Background on Power BI Report Execution

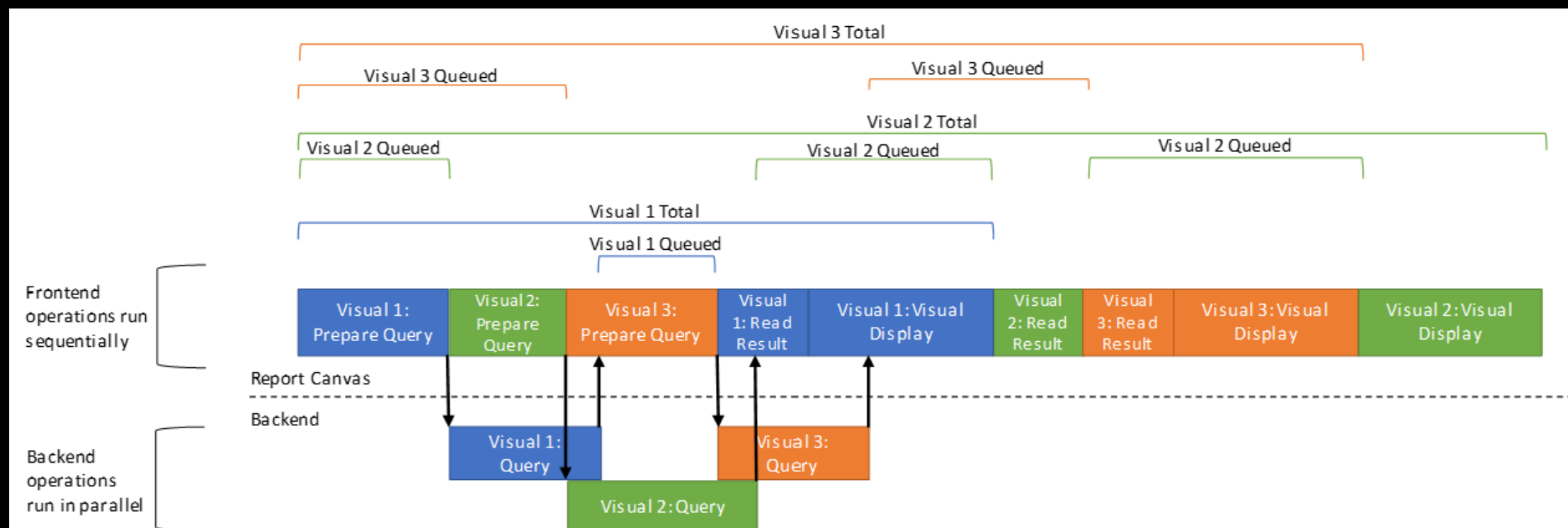
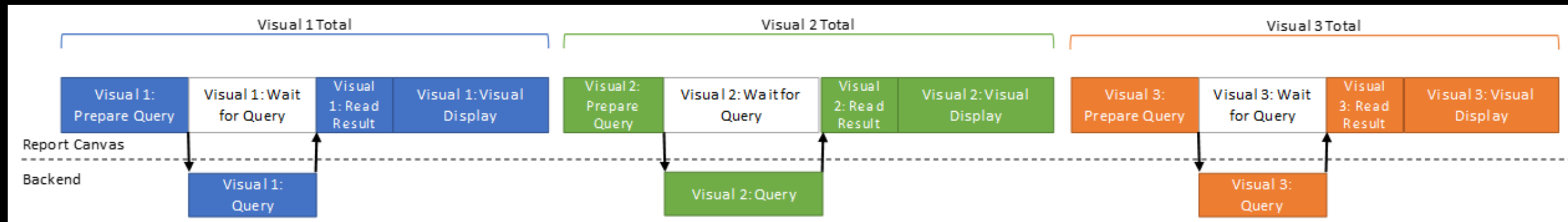
There are several major subsystems involved in executing a Power BI Report:

- **Report Canvas** provides the user interface for Power BI reports including hosting visuals and filters, managing user interactions for consuming and authoring reports, and retrieving data for display. The Report Canvas is written using web technologies and runs in web browsers or web browser components. The Report Canvas retrieves data using a high-level, internal, Power BI query language known as Semantic Query.
- **Data Shape Engine (DSE)** evaluates Semantic Queries by generating and running one, or more DAX queries against a data model hosted inside Power BI, Power BI Desktop, Azure Analysis Services, or SQL Server Analysis Services.
- **Data Model Engine (AS)** stores the data model and provides services to reports, such as DAX queries evaluation. The model may be hosted in Power BI, Power BI Desktop, Azure Analysis Services, or SQL Server Analysis Services. Depending on the data model host, a model may be tabular or multidimensional. Tabular models may contain in-memory tables, Direct Query tables, or a mix of such tables. DAX queries against tables in Direct Query mode will trigger queries to the Direct Query data source. For example, a DAX query against a Direct Query table backed by a SQL Server database will trigger one, or more, SQL queries.



Don't ignore the front end

Understanding how the front end processes updates



Don't ignore the front end

Not all visuals are created equal



Key takeaways

- Star schema is still the best default for semantic models that need to scale cleanly.
- VertiPaq rewards narrow facts, strong dimensions, and lower-cardinality columns.
- Aggregation tables, incremental refresh and hybrid tables are operational design tools, not just semantic model tricks.
- DAX scales best when model design does most of the work.
- The front end can have a huge impact on how report viewers perceive report performance
- A good redesign is measured by memory, refresh time, and concurrency together.

Discussion prompts

- Where is your biggest bottleneck today?
- Which tables need freshness vs speed?
- Which 3 columns would you remove first?
- Which measure is hiding a modeling problem?

Semantic Modeling Decision Framework

Section 6: Capacity, cost, and governance without killing agility



Capacity, cost, and governance without killing agility

Microsoft Fabric platform operating model for enterprise teams

Capacity economics

Workspace design

DevOps + guardrails

Goal: standardize the platform, decentralize delivery.



The operating question

How to preserve self-service speed without turning shared capacity into shared pain

Capacity

- Fabric cost is really shared-compute behavior under mixed workloads.
- Bursting can hide bad design; throttling reveals it.

Governance

- Governance should set guardrails around creation, assignment, promotion, and ownership.
- It should not require central review of every artifact.

Agility

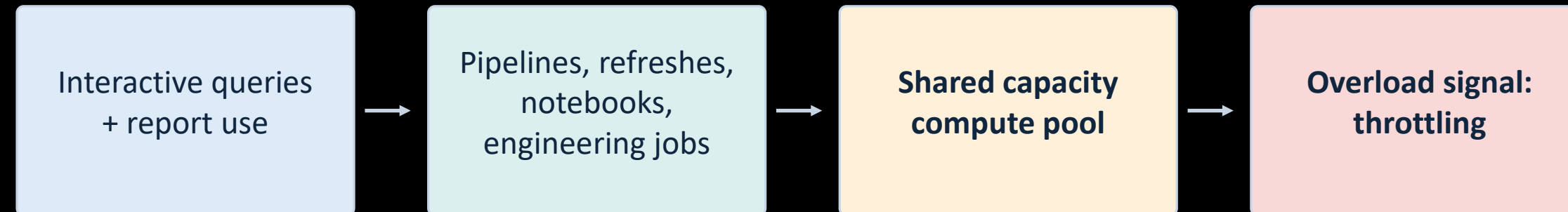
- Teams move faster when standards are reusable and production promotion is predictable.
- The platform team standardizes; domain teams deliver.

Core principle

The target state is not “less governance.” It is governance that is embedded into workspace patterns, capacity segmentation, and deployment paths.

How Fabric capacity really behaves

Economics depend on concurrency, workload mix, and throttling at the capacity boundary



Implications for cost

- Bursts can hide poor scheduling in pilots.
- Production pain appears when teams collide in the same compute window.
- Missed SLAs can cost more than the saved SKU spend.

What to watch operationally

- Top-consuming workspaces and item types
- Time-of-day spikes and month-end collisions
- When to optimize, isolate, reschedule, or resize

Bursting and Smoothing in Fabric

Bursting helps jobs finish faster. Smoothing spreads the capacity charge over time so short peaks do not immediately cause throttling.

What is bursting?

Definition: Fabric can temporarily let a workload consume more than its baseline capacity so the work completes faster.

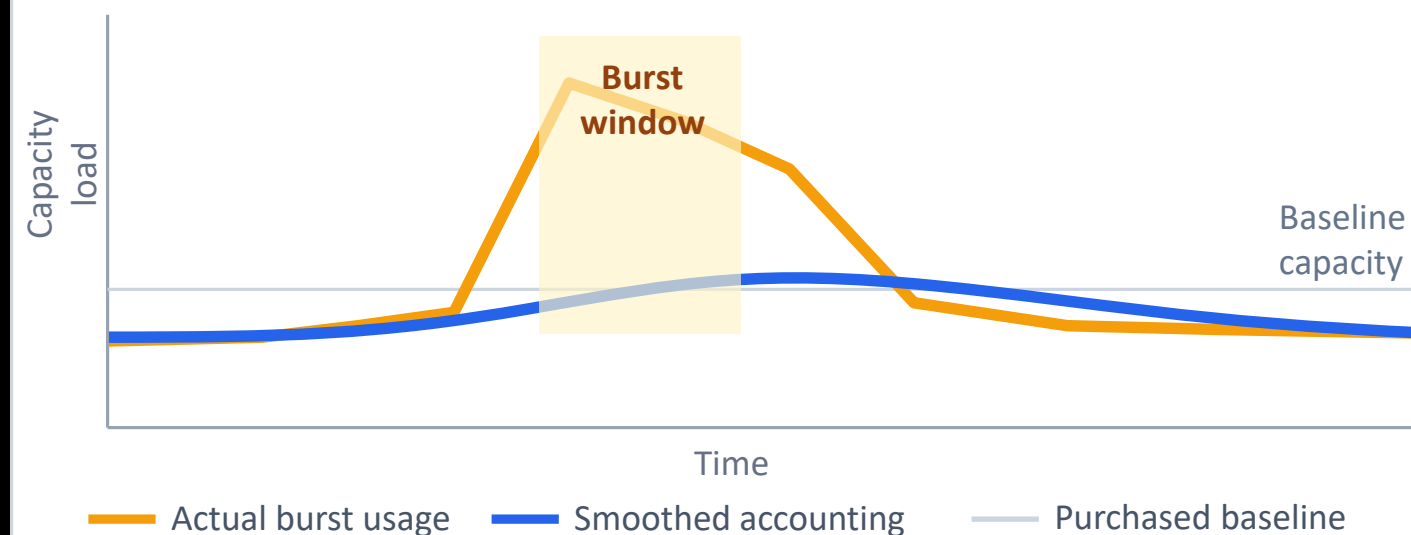
Why it exists: it absorbs short spikes instead of forcing you to size only for peak demand.

Key point: bursting changes how fast work can run; it is about performance and concurrency in the moment.

Think: "Use extra horsepower now."

How smoothing works

A bursty job uses extra compute during execution, but Fabric evaluates the capacity consumption over time so one spike is less likely to trigger throttling.



Execution can be fast now, while capacity accounting is spread forward in time.

Why this matters for teams

- Bursting and smoothing reduce the need to schedule every job around tiny peaks.
- Smoothing does not make a job slower or faster; it changes how usage is evaluated against limits.
- Sustained overuse still leads to throttling, so repeated peaks are not "free."



Throttling in Microsoft Fabric

Progressive protection when smoothed future CU consumption stays above capacity limits

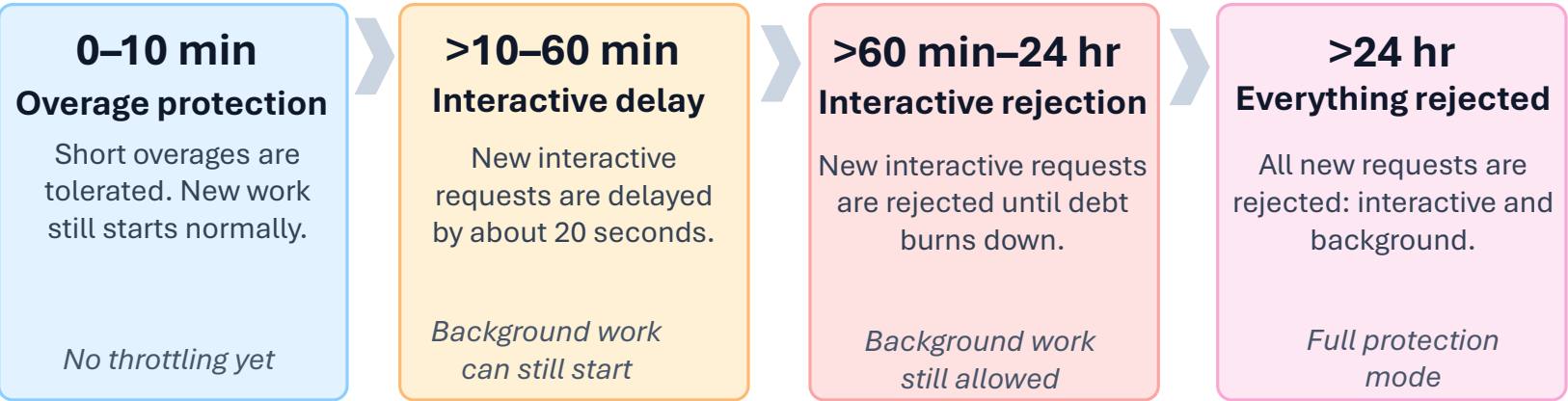
What it is

Fabric allows short spikes through **bursting** and spreads usage over time through **smoothing**. When sustained demand still stays above the SKU limit, it begins throttling new work.

Think of it as a traffic-control mechanism for shared capacity.

How throttling progresses

Thresholds are based on future capacity consumed after smoothing



How teams respond

1. Verify the pattern

Use the Capacity Metrics app to check interactive delay, interactive rejection, and background rejection trends.

2. Reduce compute pressure

Optimize semantic models, refreshes, Spark jobs, notebooks, pipelines, and expensive SQL/DAX patterns.

3. Change the capacity plan

Scale up, scale out, isolate high-priority workspaces, or use surge protection / capacity overage where appropriate.

Key takeaway: throttling does not mean “Fabric is broken”, it means sustained demand is higher than the capacity can absorb.





Isolation patterns: the trade-off is simplicity vs blast radius

A. One shared capacity

Simple

Strengths

- Easy billing story
- Few moving parts

Risks

- Noisy neighbors
- Weak accountability
- Hard to protect executive SLAs

B. Tier by criticality

Recommended default

Strengths

- Protects prod BI
- Separates experimentation
- Supports different service levels

Risks

- Needs clearer ownership model

C. Domain isolation

When scale justifies it

Strengths

- Clearer showback/chargeback
- Stronger ownership boundaries

Risks

- More capacities to govern



Estimating SKU Size and Costs

- Microsoft Fabric SKU Estimator:
<https://www.microsoft.com/en-us/microsoft-fabric/capacity-estimator>
- The quick math:
 - If you only need your capacity for 14 hours a day or less, then PAYG is cheaper than reserved capacity.
 - 168 hours in a week, 60% is 100 hours. If you don't need your capacity on weekends, you are already at 120 hours maximum needed per week. That means 20 hours per weekday PAYG is less than reserved capacity.

Fabric Power BI Capacity Planning Worksheet

Microsoft Fabric SKU Estimator (preview)

⚠ This preview of the Microsoft Fabric SKU Estimator is made available under Microsoft's [Terms of use](#). Estimates generated may not be accurate.

Tell us about your data and we'll generate a SKU recommendation based on your capacity requirements.

Data Information [Reset](#)

Total size of the data when compressed (GiB) ⓘ

Number of daily batch cycles ⓘ

Number of tables across all data sources ⓘ

Fabric usage

Select the workloads and features that you plan to use in Fabric. Some may require additional information.

☐ Data Factory ⓘ ☐ Spark Jobs ⓘ

☐ Data Warehouse ⓘ ☐ Ad-Hoc SQL Analytics ⓘ

☐ Data Science ⓘ

Power BI

☐ Power BI ⓘ ☐ Power BI Embedded ⓘ

Real-Time Intelligence

☐ Eventstream ⓘ ☐ Eventhouse ⓘ

☐ Activator Updated ⓘ

Microsoft Fabric Databases

☐ SQL database in Fabric ⓘ

Estimation

Enter the information requested. We'll estimate a Fabric SKU for you, based on your capacity requirements.

Start your free Microsoft Fabric trial now. [Learn more](#)



Workspace design is the center of governance

In Fabric, workspaces define security boundaries, capacity allocation, and operational ownership

Ownership boundary

A workspace should have a clear owner, support model, and escalation path.

Environment boundary

Dev / test / prod separation keeps promotion explicit and reduces manual edits.

Capacity boundary

Workspace placement determines which shared compute pool absorbs the load.

Security boundary

Role assignment and access groups should be managed at scale, not per person.

Good default pattern

Domain + environment workspaces (for example: Sales-Dev, Sales-Test, Sales-Prod) with shared platform workspaces for ingestion / curated data when needed.

Anti-patterns

One monster workspace per department • one workspace per developer • editing production directly • using workspace membership as the only security model

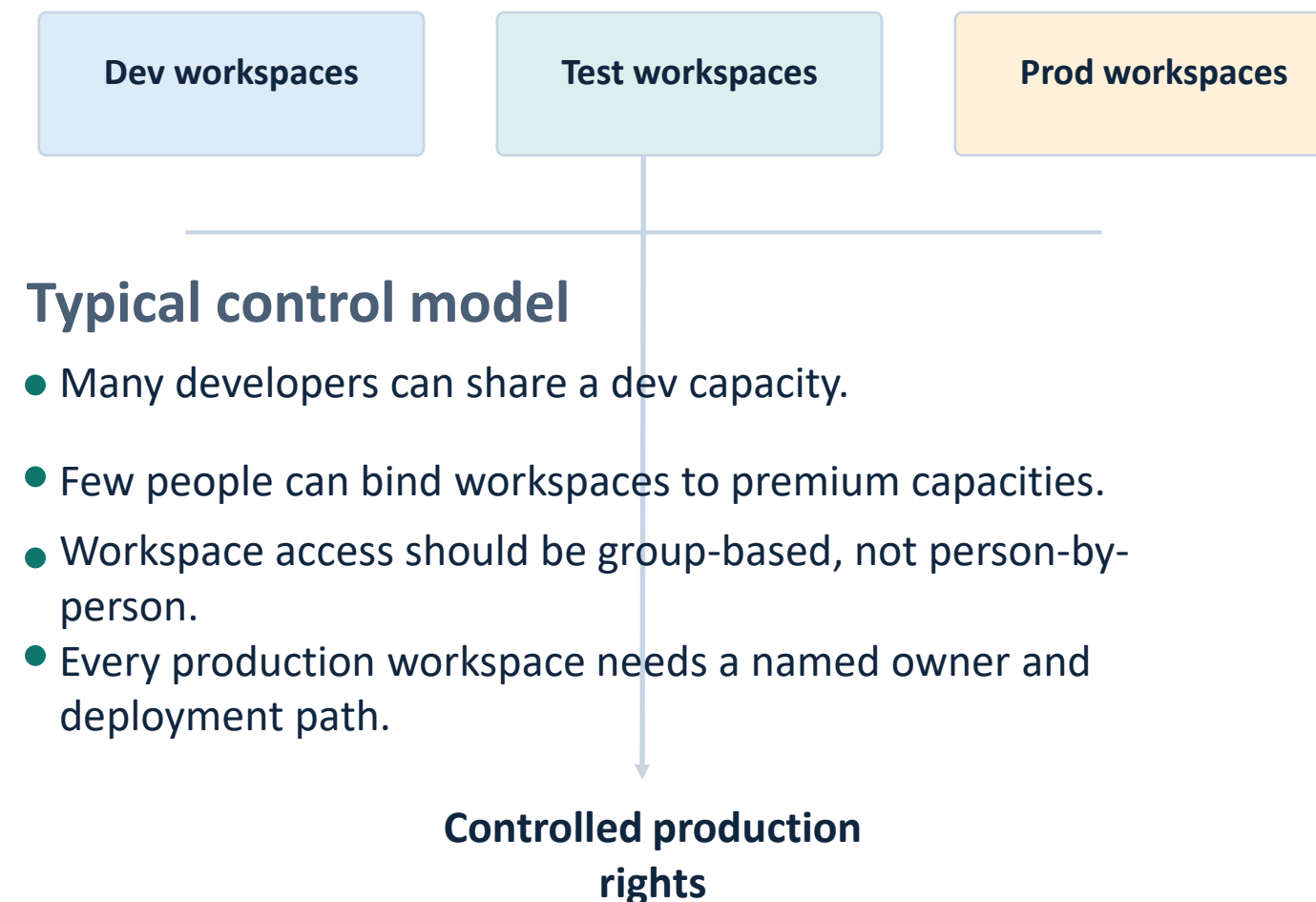


“40 devs, one capacity” can work if rights are narrower than reach

What many enterprises do by accident

- All developers share the same capacity and the same broad workspace permissions
- Manual production changes bypass Git and deployment controls
- Nobody can answer who is consuming the compute budget

What survives enterprise reality



Visibility before chargeback

Showback is the bridge between “shared platform” and accountable behavior

Capacity review cadence

Utilization Daily / weekly trend

Throttling Critical signal

Top consumers Workspace + item type

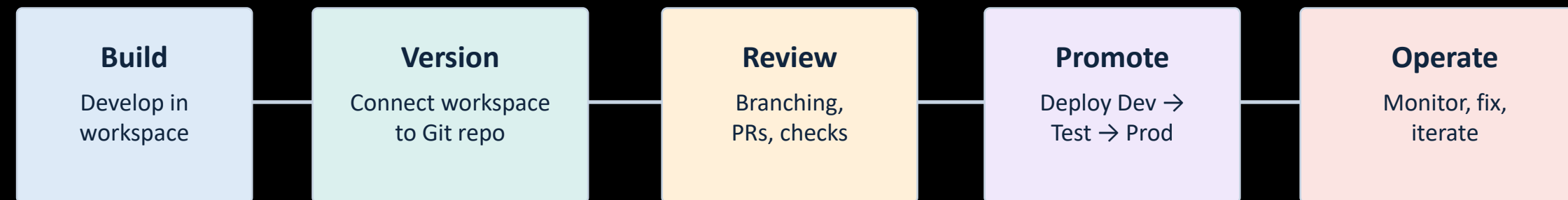
Collision windows Refresh vs interactive peaks

Governance moves that metrics should trigger

- Optimize: fix obvious workload design problems before buying more capacity
- Reschedule: move refreshes / engineering jobs away from peak interactive windows
- Isolate: carve out critical or noisy workloads into separate capacities
- Resize: add headroom only after the team can show what is driving the load
- Report: share utilization and top-consuming workspace views with business owners

Simple message for stakeholders: if the platform is shared, the evidence must be shared too.

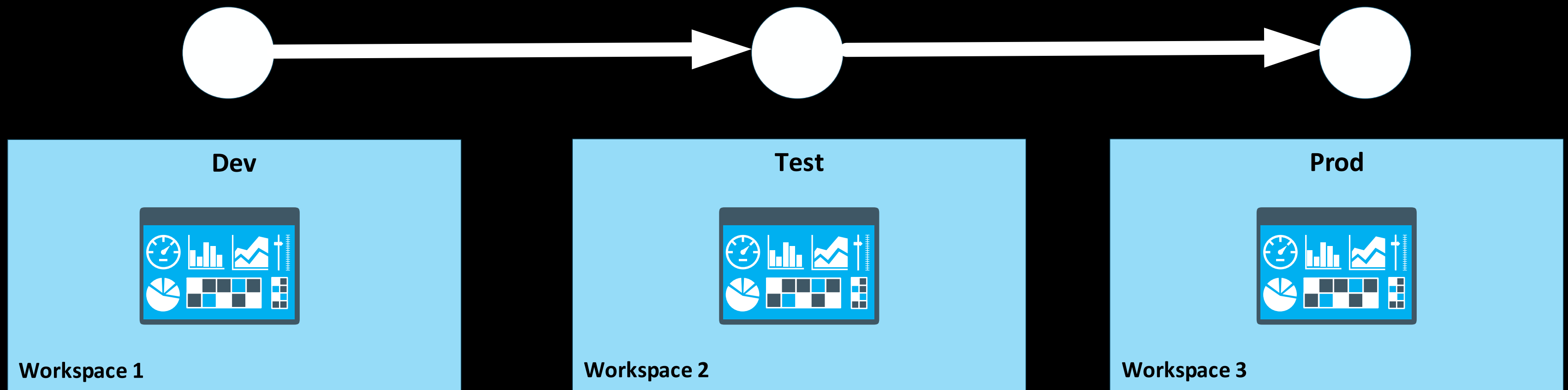
DevOps path: Git for change control, pipelines for promotion



- Git gives teams version history, branching, and code review inside Fabric.
- Deployment pipelines support 2–10 stages (default 3) for standardized promotion.
- The initial Git connection is controlled by a workspace admin.

Deploying content

Deployment Pipeline





Governance stack: make the hard rules few, clear, and automatable

1. Hard guardrails	Who can create capacities, assign workspaces, change tenant settings, connect Git, and deploy to production.
2. Default patterns	Workspace naming, environment model, branching strategy, deployment path, owner model, and monitoring expectations.
3. Managed exceptions	Temporary isolated capacity, special release cadence, or domain-specific architecture allowed with review and expiry.

This is how you keep central governance from becoming central bottlenecking.

Microsoft Fabric Governance and DevOps Checklist



Scenario exercise: design an approach that survives enterprise reality

Scenario

- 40 Fabric developers across 6 business domains
- 1 shared capacity today
- Executive dashboards with SLA expectations
- Heavy notebook / pipeline experimentation
- No Git standard and manual production changes
- Month-end slowdowns with unclear accountability

Ask each group to produce

1. Workspace strategy

2. Capacity segmentation

3. Role model

4. Git / branch strategy

5. Deployment pipeline design

6. Monitoring + showback model

7. Exception process

8. First 90-day remediation plan

Evaluation lens: protect production, support multiple teams, make cost visible, minimize manual approvals, and survive a noisy-neighbor event.

A practical 90-day target state

What a platform team can put in place quickly

Days 1–30

- Set default workspace pattern
- Identify prod-critical workloads
- Turn on Git + pipeline standards
- Start capacity metrics reviews

Days 31–60

- Separate dev from prod paths
- Create showback views by domain
- Narrow production deployment rights
- Document exception process

Days 61–90

- Isolate top noisy workloads
- Tune schedules around collision windows
- Publish operating model and RACI
- Plan next capacity split / resize

Closing message: the goal is not to slow teams down; it is to make sure one fast-moving team does not become everyone else's outage, cost spike, or audit finding.

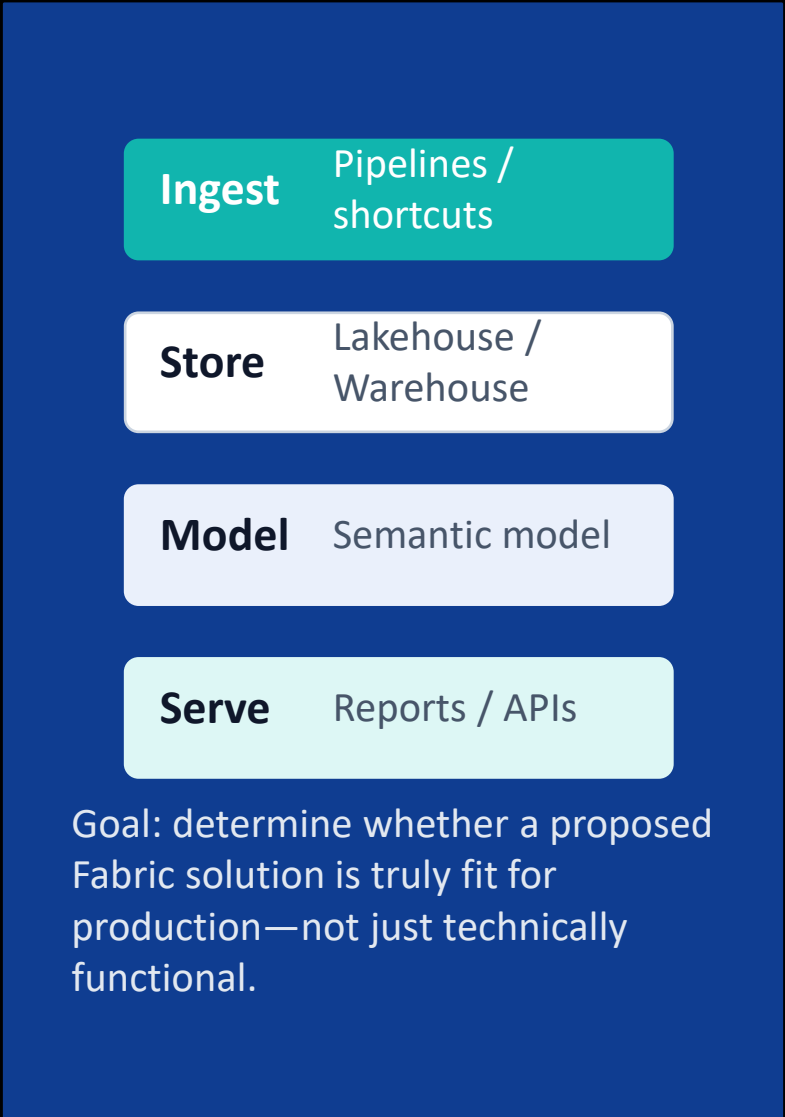
Section 7: Production readiness framework



Production readiness framework

Microsoft Fabric
architecture evaluation, risk scoring, and decision
defense

Decide + defend





What “production ready” means in Fabric

Use a go/no-go standard, not a “works on my machine” standard.

A Fabric solution is production ready when it can reliably meet business SLAs, protect data appropriately, operate within acceptable cost limits, recover from failure, and be changed safely without disrupting consumers.

Reliability

Will it keep working and recover predictably?

Security

Are access, sharing, and sensitive data controls defensible?

Operational Excellence

Can people monitor, support, and change it safely?

Cost Optimization

Can capacity and consumption stay inside guardrails?

Performance Efficiency

Does the solution meet SLAs?

Anchor the review in the five Azure Well-Architected pillars: reliability, security, cost optimization, operational excellence, and performance efficiency.





The 9 decision domains for Fabric

Review every proposed architecture across the same domains so the conversation stays structured.

1 Business Fit & Criticality	Workload criticality, data latency, SLA, impact of failure	5 DevOps & Change Safety	Git, deployment pipelines, test/prod promotion, rollback path
2 Security & Governance	Least privilege, separation of duties, workspace/domain controls, auditability	6 Architecture Clarity	Clear role for Lakehouse, Warehouse, pipelines, semantic model, serving layer
3 Reliability & Recoverability	Retries, dependency handling, recovery plan, RTO/RPO expectations	7 Operations & Observability	Monitoring, alerts, support ownership, runbooks, incident response
4 Performance & Scale	Capacity fit, concurrency, model size, refresh windows, isolation needs	8 Cost & Sustainability	SKU alignment, noisy-neighbor risk, consumption pattern, FinOps visibility
9 Decision Governance		Design decisions, alternatives and risks are clearly documented	



Architecture evaluation checklist

A strong review checks evidence, not opinions.

Solution + environment

- Business owner and SLA identified
- Dev / test / prod strategy exists
- Workspace structure is intentional
- Capacity placement documented

Data + security

- Sources and ownership documented
- Lakehouse / Warehouse / model roles are clear
- Sensitive data handling defined
- Access model is least-privilege

Scale + operations

- Volume and concurrency assumptions stated
- Refresh window is realistic
- Monitoring and alerts are defined
- Runbook + support owner exist

Delivery + cost

- Git + deployment path are repeatable
- Rollback path exists
- Capacity estimate is documented
- Trade-offs and approvals are written down



Risk scoring model: simple enough to use live

Score each domain, weight the critical ones, and prevent weak designs from “averaging out.”

Scoring scale

0

Not addressed

1

Weak / ad hoc

2

Partially addressed

3

Production-acceptable

4

Strong / well-controlled

Suggested weighting

Business Fit & Criticality	10%
Security and Governance	20%
Reliability and Recoverability	20%
Performance and Scale	15%
DevOps and Change Safety	10%
Architectural Clarity	10%
Operations and Observability	5%
Cost and Sustainability	5%
Decision Governance	5%

Decision thresholds

3.5–4.0

Production ready

2.8–3.4

Ready with conditions

2.0–2.7

Pilot only / not yet prod

< 2.0

Redesign required

Hard stop rules: do not approve if Security < 3, Reliability < 3, or DevOps < 2.5.

Fabric Production Ready Scoring Worksheet

Pg: 71



Decide + defend: how to justify the architecture

Use the same facts, but translate them for the audience.

Technical stakeholders

- State the constraints: workload criticality, latency, scale, security, cost ceiling.
- Name the alternatives considered and why they were rejected.
- Be explicit about trade-offs: isolation vs. cost, speed vs. control, flexibility vs. standardization.
- Document residual risks and the mitigation plan.

Executive stakeholders

- Translate design choices into continuity, trust, accountability, and cost predictability.
- Explain what failure would mean for the business and how the design reduces that exposure.
- Show where added cost buys risk reduction or service stability.
- Make the approval decision and conditions easy to understand.

Defense formula: decision + rationale + key trade-offs + risks accepted + actions required before approval



Require decision records, not just diagrams

Fabric designs are easier to defend when major choices are written down before they become hard to reverse.

Architecture decision record (ADR) template

- Decision
- Context
- Alternatives considered
- Consequences / trade-offs
- Owner + review date

Typical Fabric decisions worth recording

- Why Lakehouse instead of Warehouse for this workload?
- Why Direct Lake vs. Import vs. DirectQuery?
- Why one shared capacity vs. isolated capacity for production?
- Why centralized workspace model vs. domain-per-business-unit?
- Why notebooks vs. pipelines vs. dataflows for transformation?

Architecture Decision Record Template Pack



Common “not production ready” signals in Fabric

These are the patterns reviewers should flag immediately.

- ❗ POC architecture promoted directly into production
- ❗ Dev / test / prod separation is missing or symbolic only
- ❗ Workspace layout mirrors org-chart chaos instead of lifecycle control
- ❗ Deployment pipelines exist, but release discipline does not
- ❗ Shared capacity with no isolation strategy for critical workloads
- ❗ Monitoring depends on a person noticing something broke
- ❗ Direct Lake chosen without validating model behavior and query fit
- ❗ Cost is assumed to be “fine” instead of estimated and governed
- ❗ No clear owner for semantic models and certified reporting assets



Final exercise: assess and defend a proposed architecture

Use the checklist and scoring model to determine whether the design is production ready.

Scenario

- One shared Fabric capacity supporting multiple business units
- 40 developers; mix of self-service and enterprise reporting
- Shared Lakehouse feeding several executive dashboards
- Limited deployment pipelines and no strict dev / test / prod separation
- No documented concurrency assumptions or formal monitoring model

Team output

- Is it production ready? If yes, for which workload tier?
- Top 5 risks and the score for each decision domain
- What must change before approval?
- What can be accepted temporarily as residual risk?
- Defend the decision to the architecture board, governance lead, and CIO/CFO

Important nuance: A design might be acceptable for departmental BI but not for enterprise-certified or executive-critical reporting.

Recommended final decision statement

Give reviewers a reusable template for approval, conditional approval, or rejection.

Decision: Conditionally approved for production

- Scope of approval: Departmental reporting only; not approved for enterprise-certified or executive-critical reporting.
- Reasons: The core Fabric architecture is viable, but current gaps in capacity isolation, deployment control, and monitoring create unacceptable operational risk for high-criticality workloads.
- Required actions before full approval: establish production isolation, define release process, implement monitoring and alerting, document support ownership, and validate concurrency assumptions.
- Residual risks accepted: moderate cost variability and manual operating overhead during early adoption.

Microsoft Fabric Production Readiness Toolkit

Section 8: Q&A + next steps